Assignment 3

Assignment Questions:

Use make file and implement. You are given m input vectors, each with n numbers. Your task is to calculate the correlation between every pair of input vectors.

Interface

You need to implement the following function:

```
void correlate(int ny, int nx, const float* data, float* result)
```

Here data is a pointer to the input matrix, with ny rows and nx columns. For all 0 <= y < ny and 0 <= x < nx, the element at row y and column x is stored in data[x + y*nx]. The function has to solve the following task: for all i and j with 0 <= j <= i < ny, calculate the **correlation coefficient** between row i of the input matrix and row j of the input matrix, and store the result in result[i + j*ny].

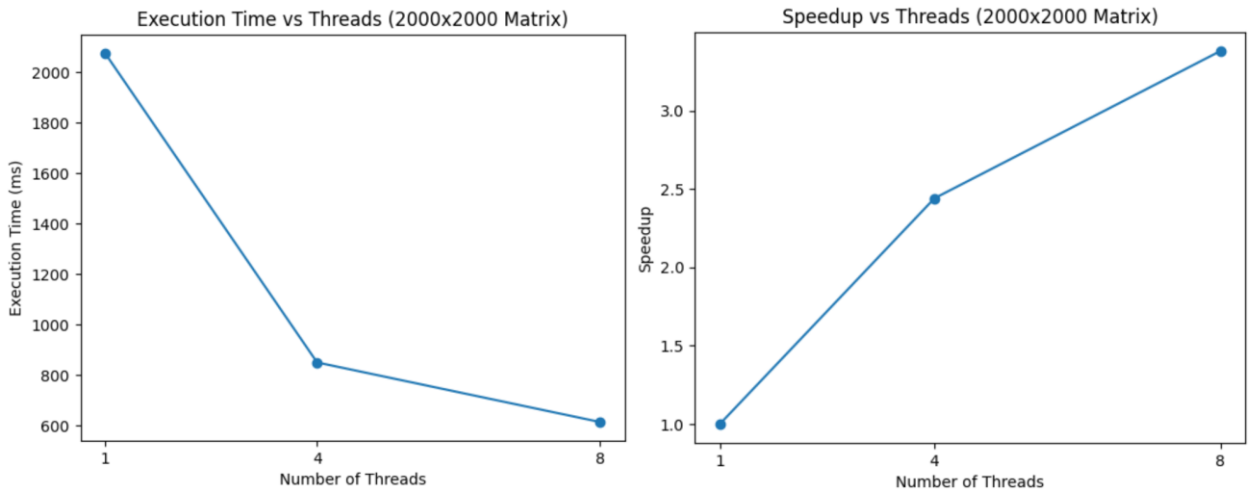
Make sure there is main.cpp corelate.cpp and makefile

1. Implement a simple sequential baseline solution. Do not try to use any form of parallelism yet; try to make it work correctly first. Please do all arithmetic with double-precision floating point numbers.
2. Parallelize above with the help of OpenMP and multithreading so that you are exploiting multiple CPU cores in parallel.
3. Using all resources that you have in the CPU, solve the task as fast as possible. You are encouraged to exploit instruction-level parallelism, multithreading, and vector instructions whenever possible, and also to optimize the memory access pattern.
4. Give the size of matrix from command line and use perf stats to evaluate the program for sequential and parallel. Increase the number of threads and size of matrix.

RESULT:

Matrix Size	Threads	Time (ms)	Speedup
1000×1000	1	477.812	1.00×
1000×1000	4	199.004	2.40×
2000×2000	1	2074.03	1.00×
2000×2000	4	848.80	2.44×
2000×2000	8	612.87	3.38×

In this experiment, the pairwise correlation of matrix rows was implemented using a sequential baseline and then parallelized with OpenMP. The matrix size and number of threads were provided through command-line arguments and performance was evaluated using `perf stat`. Results showed that increasing the number of threads significantly reduced execution time and improved CPU utilization (from ~2074 ms with 1 thread to ~613 ms with 8 threads for a 2000×2000 matrix). The nearly constant page faults indicate a compute-intensive workload. This demonstrates that utilizing multithreading and available CPU resources effectively improves performance.



OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ make
g++ -O3 -std=c++11 -Wall -fopenmp -c main.cpp -o main.o
g++ -O3 -std=c++11 -Wall -fopenmp -c correlate.cpp -o correlate.o
g++ -O3 -std=c++11 -Wall -fopenmp -o correlate main.o correlate.o
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ ./correlate 1000 1000 1
Matrix: ny=1000, nx=1000
Threads: 1
Time taken: 477.812 ms
Sample output (0,0): 1
Sample output (1,0): -0.00672736
Sample output (1,1): 1
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ ./correlate 1000 1000 4
Matrix: ny=1000, nx=1000
Threads: 4
Time taken: 199.004 ms
Sample output (0,0): 1
Sample output (1,0): -0.00672736
Sample output (1,1): 1
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ ./correlate 2000 2000 8
Matrix: ny=2000, nx=2000
Threads: 8
Time taken: 934.062 ms
Sample output (0,0): 1
Sample output (1,0): 0.0164815
Sample output (1,1): 1
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ /usr/lib/linux-tools/6.8.0-90-generi
c/perf stat -e task-clock,context-switches,cpu-migrations,page-faults ./correlate 2
000 2000 1
Matrix: ny=2000, nx=2000
Threads: 1
Time taken: 2074.03 ms
Sample output (0,0): 1
Sample output (1,0): 0.0164815
Sample output (1,1): 1

 Performance counter stats for './correlate 2000 2000 1':

          2122.30 msec task-clock:u                     #    0.988 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
             7959      page-faults:u                    #    3.750 K/sec

       2.147206271 seconds time elapsed

       2.107641000 seconds user
       0.015906000 seconds sys
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ /usr/lib/linux-tools/6.8.0-90-generic
/perf stat -e task-clock,context-switches,cpu-migrations,page-faults ./correlate 200
0 2000 4
Matrix: ny=2000, nx=2000
Threads: 4
Time taken: 848.802 ms
Sample output (0,0): 1
Sample output (1,0): 0.0164815
Sample output (1,1): 1

 Performance counter stats for './correlate 2000 2000 4':

          3408.45 msec task-clock:u                     #    3.715 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
             7963      page-faults:u                    #    2.336 K/sec

       0.917391856 seconds time elapsed

       3.382769000 seconds user
       0.019598000 seconds sys
```

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment3$ /usr/lib/linux-tools/6.8.0-90-generic
/perf stat -e task-clock,context-switches,cpu-migrations,page-faults ./correlate 200
0 2000 8
Matrix: ny=2000, nx=2000
Threads: 8
Time taken: 612.869 ms
Sample output (0,0): 1
Sample output (1,0): 0.0164815
Sample output (1,1): 1

 Performance counter stats for './correlate 2000 2000 8':

          4842.62 msec task-clock:u                     #    7.187 CPUs utilized

                0      context-switches:u               #    0.000 /sec

                0      cpu-migrations:u                 #    0.000 /sec

             7972      page-faults:u                    #    1.646 K/sec


       0.673776466 seconds time elapsed

       4.826448000 seconds user
       0.027512000 seconds sys
```

Vaani Prashar
3C16
102303159